

5. Comunicação Baseada em Eventos: MQTT

Objetivos de aprendizagem

- Explicar publish/subscribe, broker, tópico e sessão.
- Distinguir QoS 0, 1 e 2 sem confundir entrega com processamento de negócio.
- Projetar hierarquias de tópicos e usar wildcards.
- Executar Mosquitto e implementar publishers/subscribers em Python.
- Analisar retained messages, Last Will, duplicação e indisponibilidade.

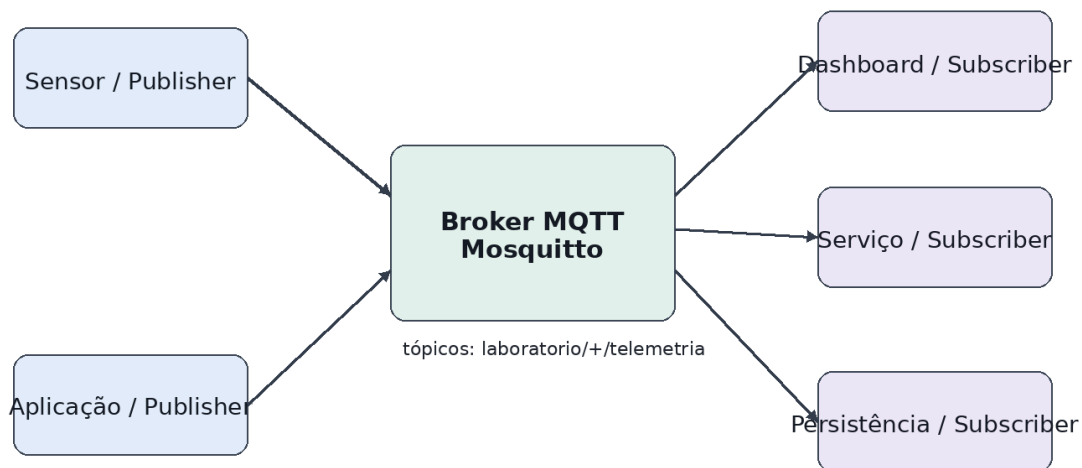


Figura 4 - Arquitetura publish/subscribe utilizando um broker MQTT

5.1 Por que eventos?

Na comunicação direta, o produtor precisa conhecer o endpoint do consumidor. Em publish/subscribe, o produtor publica uma mensagem em um tópico administrado pelo broker. Consumidores interessados assinam tópicos e recebem as mensagens conforme as regras do protocolo. Isso reduz o acoplamento espacial entre produtor e consumidor e facilita adicionar novos consumidores sem alterar o produtor.

5.2 Componentes

- Broker: recebe publicações, aplica regras de sessão/QoS e encaminha mensagens aos assinantes.
- Publisher: cliente que publica mensagens.
- Subscriber: cliente que registra interesse em um ou mais tópicos.
- Tópico: string hierárquica usada para roteamento lógico.
- Payload: conteúdo da mensagem, cujo formato é definido pela aplicação.

5.3 Tópicos e wildcards

Uma taxonomia bem desenhada facilita autorização, observabilidade e expansão. Exemplo: campus/blocoA/sala101/temperatura. O wildcard + substitui um nível e # cobre níveis restantes. Um subscriber campus/+/+/temperatura pode receber temperaturas de diferentes blocos e salas.

5.4 Qualidade de Serviço (QoS)

Tabela 9 - Níveis de QoS do MQTT

QoS	Objetivo de entrega	Implicação	Cuidado
0	No máximo uma vez	Menor overhead; sem confirmação de entrega	Perdas são possíveis
1	Pelo menos uma vez	Reenvio até confirmação	Duplicatas são possíveis
2	Exatamente uma vez no fluxo MQTT	Handshake adicional	Maior custo; não equivale a “efeito de negócio exatamente uma vez”

QoS descreve a entrega da mensagem entre clientes e broker conforme o protocolo. Mesmo com QoS 2, um consumidor pode executar uma ação não idempotente duas vezes se sua lógica de processamento falhar em um ponto inadequado. “Exactly once” de transporte não é garantia automática de “exactly once” de negócio.

5.5 Retained messages e Last Will

Uma mensagem retained é armazenada pelo broker como o último valor conhecido de um tópico e é entregue a novos assinantes. É útil para estado atual, mas não substitui um histórico. Last Will and Testament permite que o broker publique uma mensagem definida previamente quando detecta desconexão anormal do cliente, sendo útil para indicar disponibilidade.

5.6 Laboratório com Mosquitto

compose.yaml

```
services:
  mosquitto:
    image: eclipse-mosquitto:2
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
```

mosquitto.conf

```
listener 1883
allow_anonymous true
```

Uso didático

allow_anonymous true simplifica o laboratório local. Em produção, autenticação, autorização por tópico e TLS devem ser considerados. Não exponha esse broker didático diretamente à Internet.

Terminal

```
docker compose up -d
docker compose logs -f mosquitto
```

Teste pelo terminal

Terminal

```
# Terminal 1
mosquitto_sub -h localhost -t 'laboratorio/+/telemetria' -v

# Terminal 2
mosquitto_pub -h localhost -t 'laboratorio/sensor1/telemetria' \
-m '{"temperatura":24.3,"unidade":"C"}' -q 1
```

Publisher em Python

Terminal

```
pip install paho-mqtt
```

publisher.py

```
import json, random, time
import paho.mqtt.client as mqtt

BROKER = "localhost"
TOPICO = "laboratorio/sensor1/telemetria"

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="sensor1")
client.connect(BROKER, 1883, 60)
client.loop_start()

try:
    for seq in range(1, 21):
        payload = {
            "seq": seq,
            "temperatura": round(random.uniform(20, 30), 2),
            "unidade": "C",
        }
        info = client.publish(TOPICO, json.dumps(payload), qos=1)
        info.wait_for_publish()
        print("publicado", payload)
        time.sleep(1)
finally:
    client.loop_stop()
    client.disconnect()
```

Subscriber em Python

subscriber.py

```
import json
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, reason_code, properties):
    print("conectado:", reason_code)
    client.subscribe("laboratorio/+/telemetria", qos=1)

def on_message(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode())
        print(msg.topic, data["seq"], data["temperatura"])
    except (json.JSONDecodeError, KeyError) as exc:
        print("mensagem inválida:", exc)

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="painel")
client.on_connect = on_connect
client.on_message = on_message
client.connect("localhost", 1883, 60)
client.loop_forever()
```

5.7 Experimentos

- Compare QoS 0 e QoS 1 sob reinício do broker ou desconexão temporária.

- Publique uma mensagem retained de estado e conecte um subscriber somente depois.
- Implemente LWT no publisher e finalize o processo abruptamente.
- Crie dois subscribers com filtros diferentes e mostre que o publisher não conhece seus endereços.
- Aumente a taxa de publicação e introduza processamento lento no subscriber; observe fila, atraso e possível backpressure.
- Inclua identificador único e número de sequência em cada mensagem para detectar perdas, duplicação ou reordenação.

5.8 Segurança MQTT

Em redes reais, o broker é infraestrutura crítica. Controles incluem TLS, autenticação, ACLs por tópico, isolamento de rede, limite de conexões e taxa, logs e rotação de credenciais. Wildcards amplos podem permitir que um cliente receba dados além do necessário. A taxonomia de tópicos também é uma decisão de segurança.

AP3 - Sistema distribuído orientado a eventos com MQTT

Atividade prática

Construa um sistema com pelo menos dois publishers e dois tipos de subscribers. O foco é demonstrar desacoplamento, roteamento por tópicos, comportamento sob indisponibilidade e análise de QoS.

Objetivos da atividade

- Projetar taxonomia de tópicos.
- Implementar produtores e consumidores independentes.
- Experimentar QoS, retained e/ou LWT.
- Analisar perdas, duplicação, atraso e disponibilidade.

Requisitos obrigatórios

- Broker Mosquitto local ou equivalente.
- Pelo menos quatro processos clientes no cenário completo.
- Payload estruturado contendo identificador e sequência.
- Uso de pelo menos dois níveis de QoS em experimentos.
- Um experimento com consumer ou broker temporariamente indisponível.

Roteiro mínimo

1. Defina domínio e tópicos.
2. Implemente publishers e subscribers separadamente.
3. Registre 50 ou mais mensagens por cenário experimental.
4. Introduza falha ou desconexão controlada.
5. Compare resultados e apresente evidências.

Entregáveis

- Código e arquivos de configuração.
- README de execução.
- Tabela de resultados por cenário.
- Logs demonstrando sequência e eventual duplicação/perda.
- Diagrama da arquitetura e demonstração.

Questões para análise

1. O que exatamente foi desacoplado pelo broker?
2. QoS escolhido é suficiente para o requisito do domínio?
3. Como o sistema identifica mensagem duplicada?
4. Quando retained é apropriado e quando é perigoso?
5. O que acontece se o broker se tornar ponto único de falha?

Critérios de avaliação

Tabela 10 - Critérios de avaliação da AP3

Critério	Evidência esperada
Arquitetura de tópicos	Hierarquia coerente e extensível.
Implementação	Produtores e consumidores independentes funcionam.
Experimentos	QoS e falhas foram comparados com evidências.
Análise	Discussão distingue entrega de transporte e efeito de negócio.
Demonstração	Cenário reproduzível e bem explicado.

Checklist antes da apresentação

- ☐ Todos os processos possuem IDs claros.
- ☐ As mensagens possuem sequência.
- ☐ Foram executados cenários com diferentes QoS.
- ☐ Há pelo menos um teste de indisponibilidade.
- ☐ A análise explica o comportamento observado.

Desafios opcionais

- Adicione autenticação e ACL por tópico.
- Implemente persistência do subscriber e reprocessamento idempotente.
- Utilize MQTT 5 para propriedades/correlation data em request-response.

5.9 Sessões, reconexão e entrega após indisponibilidade

O comportamento de mensagens quando um cliente desconecta depende da versão do MQTT, das opções de sessão e do QoS. Em termos conceituais, uma sessão persistente permite que o broker mantenha informações de assinatura e, conforme configuração, mensagens pendentes para um cliente que retorna. Isso reduz acoplamento temporal, mas cria estado no broker e exige limites de expiração para evitar crescimento indefinido.

Reconexão automática precisa de backoff. Um dispositivo que perde acesso ao broker junto com milhares de outros pode gerar tempestade de reconexão quando a rede volta. Jitter e limites progressivos distribuem tentativas no tempo. O cliente também deve decidir o que fazer com dados produzidos offline: descartar, armazenar localmente, agregar ou transmitir depois.

5.10 Envelope de mensagem e evolução de esquema

Como MQTT não impõe formato de payload, aplicações maduras definem um envelope mínimo. Campos úteis incluem `message_id`, `producer_id`, `sequence`, `timestamp`, `schema_version` e `data`. O envelope facilita deduplicação, rastreamento e evolução. O `timestamp` deve ser interpretado com cuidado: relógios de dispositivos podem estar incorretos, portanto o instante de recepção no backend pode precisar ser registrado separadamente.

exemplo de envelope JSON

```
{
  "message_id": "8f0e...",
  "producer_id": "sensor-17",
  "sequence": 1842,
  "schema_version": 1,
  "timestamp": "2026-08-20T12:00:00Z",
  "data": {"temperatura": 24.7, "unidade": "C"}
}
```

5.11 Shared subscriptions e escala de consumidores

Em alguns brokers e versões do protocolo, `shared subscriptions` permitem que um grupo de consumidores compartilhe mensagens de uma assinatura, distribuindo carga em vez de entregar cada mensagem a todos. Conceitualmente, isso muda `fan-out` para `competing consumers`. A decisão deve preservar a semântica: se cada consumidor precisa observar todos os eventos, compartilhamento não serve; se queremos processar cada evento uma vez por um worker do grupo, ele é adequado.

5.12 Ordem e particionamento lógico

Não presuma uma ordem global entre mensagens produzidas por vários publishers. Mesmo quando um fluxo individual preserva determinada ordenação, reconexões, múltiplas rotas e processamento concorrente podem alterar observação no consumidor. Se a ordem de eventos de uma entidade é necessária, use `sequence` por produtor ou por entidade e trate lacunas explicitamente.